



LABORATORIO 9

AJAX y consumo de API REST con Express

1. Introducción

En este laboratorio se integran **AJAX y APIs REST** en la aplicación web. El objetivo es que los alumnos aprendan a realizar peticiones asíncronas desde el cliente para consultar y actualizar datos en el servidor Express, mejorando la experiencia de usuario sin recargas completas de página.

Se implementará un **endpoint REST** en Express para gestionar reservas y se consumirán esos datos con **fetch()** desde el frontend.

2. Objetivos del laboratorio

- Crear endpoints REST en Express que devuelvan JSON.
- Consumir datos con `fetch()` desde el cliente.
- Mostrar resultados dinámicamente en el DOM.
- Realizar inserciones (POST) mediante AJAX.
- Gestionar errores y mostrar mensajes al usuario.
- Preparar el sistema para evolucionar hacia una RIA (Rich Internet Application).

3. Tareas a realizar

Tareas básicas (obligatorias)

1. Crear un endpoint en Express `/api/vehiculos` que devuelva la lista de vehículos en **JSON**.
2. En el cliente (`vehiculos.html`), usar `fetch()` para cargar los vehículos al iniciar la página y mostrarlos en una tabla dinámica (manipulación DOM).
3. Implementar un formulario de **nueva reserva** que envíe datos mediante `fetch()` (POST a `/api/reservas`).
4. En el servidor, guardar las reservas en un array en memoria y devolver un mensaje JSON de éxito/error.

Tareas intermedias

5. Crear una página `reservas.html` que:
 - Haga una petición GET a `/api/reservas`.
 - Muestre dinámicamente la lista de reservas.
6. Añadir validación en servidor: no se puede reservar una fecha anterior al día actual.



7. Mostrar mensajes de error en el cliente si la reserva no es válida.
8. Actualizar la tabla de reservas en el cliente automáticamente al añadir una nueva reserva, sin recargar la página.

Tareas avanzadas

9. Añadir un filtro dinámico en el cliente:
 - Ejemplo: desplegable para seleccionar tipo de vehículo → actualiza la tabla llamando a `/api/vehiculos?tipo=coche`.
10. Implementar eliminación de reservas desde el cliente (botón que mande DELETE a `/api/reservas/:id`).
11. Manejar códigos de estado HTTP (201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error) y mostrar mensajes adecuados.
12. Mostrar una animación o indicador de “cargando...” mientras se esperan las respuestas AJAX.

4. Resultado esperado

Al finalizar el laboratorio, los alumnos tendrán una aplicación que combina **Express como API REST** y **JavaScript en el cliente con fetch()** para:

- Consultar vehículos y reservas en JSON.
- Insertar nuevas reservas sin recargar la página.
- Gestionar errores en servidor y cliente.
- Filtrar y eliminar datos mediante AJAX.

Con este laboratorio, la aplicación ya se convierte en una **RIA (Rich Internet Application)** con comunicación cliente–servidor dinámica, tal como exige la práctica obligatoria.